



Aug 19, 2026

Testing Report

Key Insights From TestSprite's Autonomous AI Testing Agent

Report Contents

1	Executive Summary	03
2	Frontend UI Test Results	03

18

Test Runs

100.0%

Pass Rate

1

Issues

2h

Saved By TestSprite

Table of Contents

- 3 Executive Summary
 - 3 High-Level Overview
 - 3 Key Findings

- 3 Frontend UI Test Results
 - 3 Test Coverage Summary
 - 4 Test Execution Summary
 - 5 Test Execution Breakdown

Executive Summary

1 High-Level Overview

OVERVIEW	
Project Name	Operator Data Int Check
Project Type	Frontend
Total Test Cases	18
Base URL	https://operator-data-integrity-checker.streamlit.app/
Pass / Fail / Blocked	Frontend: 17 Passed / 0 Failed / 1 Blocked

2 Key Findings

Test Summary

The frontend run looks strong overall: 17 tests passed and 1 test was blocked, with no outright failures. The score is computed on the executable subset only, excluding the blocked case. The passing coverage indicates the main user journeys are working well: demo data loads, column mapping auto-detects, thresholds can be adjusted, and anomaly results render correctly. The one blocked test does not indicate a product defect by itself, but it does reduce confidence in the app’s reactivation/sleep-state flow because the harness could not find any wake control or sleeping-state message.

What could be better

The only issue surfaced was a blocked reactivation flow in `Return to the app after sleep state`. The harness reported: "The reactivation flow could not be executed because no sleep/wake control was found on the app." It also observed that "The app loaded its main checker interface and demo dataset" and that "The Column Mapping section shows `\"Demo columns mapped automatically.\"` and is already present." That suggests the app may not expose the expected sleep/wake UI, or the test assumption is stale. Everything else in the core data import, preview, mapping, threshold, and results flow passed.

Recommendations

Confirm whether the app is supposed to support a sleep/wake state in this environment. If yes, add a visible wake control or a clear sleeping-state banner so the automation can re-enter the app. If not, retire or rewrite the blocked test so it targets a real user flow. Then re-run the affected test (`Return to the app after sleep state`) plus nearby coverage around `Inspect demo data after waking the app` and `View loaded demo records in the preview table` to verify the UI state transitions remain stable.

Frontend UI Test Results

1 Test Coverage Summary

This report summarizes the frontend UI testing results for the application. TestSprite's AI agent automatically generated and executed tests based on the UI structure, user interaction flows, and visual components. The tests aimed to validate core functionalities, visual correctness, and responsiveness across different states.

CATEGORY	TEST CASES	PASS/FAIL RATE
Detection Settings	3	3 Pass / 0 Fail
Data Source & Import	4	4 Pass / 0 Fail
Results & Flagging	4	4 Pass / 0 Fail
Data Preview	3	3 Pass / 0 Fail
Column Mapping	4	3 Pass / 0 Fail / 1 Blocked

Note

The test cases were generated using real-time analysis of the application's UI hierarchy and user flows. Some visual and functional validations were adapted dynamically based on runtime DOM changes.

2 Test Execution Summary

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
Detection Settings			
Use a stricter detection threshold before running the checker	Verify lowering the speed threshold produces an analysis based on stricter detection settings.	Medium	PASSED
Keep the default detection threshold and run the checker	Verify the checker runs successfully with the default detection settings selected on load.	High	PASSED
Adjust detection threshold and run checker	Verify the checker uses an updated speed threshold when the user changes detection settings and runs the analysis.	High	PASSED
Data Source & Import			
Upload a local operator file	Verify a user can switch to file upload, provide a dataset file, and see the imported records populated in the preview.	High	PASSED
Change detection threshold before import analysis	Verify a user can adjust the detection setting after the dataset loads and the app remains ready to analyze the imported records.	Medium	PASSED
Load demo operator dataset	Verify the app loads the built-in demo dataset and shows a populated data preview for analysis.	High	PASSED
Review automatic column mapping after import	Verify imported operator data is automatically mapped into the app's analysis fields after a dataset is loaded.	Medium	PASSED
Results & Flagging			
Review detected anomalies in the summary view	Validates that the anomaly results view shows the main detection summary after running on demo data.	High	PASSED
View anomaly details for a suspicious record	Validates that selecting a suspicious record reveals its anomaly details in the results area.	Medium	PASSED
Review results after changing the detection threshold	Validates that updating the detection threshold refreshes the flagged results shown on the page.	Medium	PASSED
Inspect flagged records in the results table	Validates that records identified as suspicious are visible in the results table after loading the demo dataset.	High	PASSED

Data Preview				
View loaded demo records in the preview table	Validates that the demo dataset loads and is visible in the data preview table.	High	PASSED	
Inspect demo data after waking the app	Validates that the preview remains available after the app resumes from inactivity.	Low	PASSED	
Review the visible preview columns	Validates that the loaded data preview shows the expected record fields for inspection.	Medium	PASSED	
Column Mapping				
Review mapped columns with imported data	Verify that an uploaded dataset is presented in the mapping workflow so the user can review field associations before analysis.	Medium	PASSED	
Auto-map demo columns	Verify the app automatically maps the demo dataset fields to the expected operator schema.	High	PASSED	
Upload a dataset for mapping	Verify that selecting a file source and uploading a dataset leads to column mapping for the imported file.	High	PASSED	
Return to the app after sleep state	Verify that the app can be reactivated after entering the sleep state so column mapping can continue.	Low	BLOCKED	

3 Test Execution Breakdown

Failed & Blocked Test Details

Return to the app after sleep state

ATTRIBUTES	
Status	BLOCKED
Priority	Low
Description	Verify that the app can be reactivated after entering the sleep state so column mapping can continue.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/20260819-134902-311feccf.webm

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

16
17
18
19
20
21
22
23
24
25
26
27
28

29

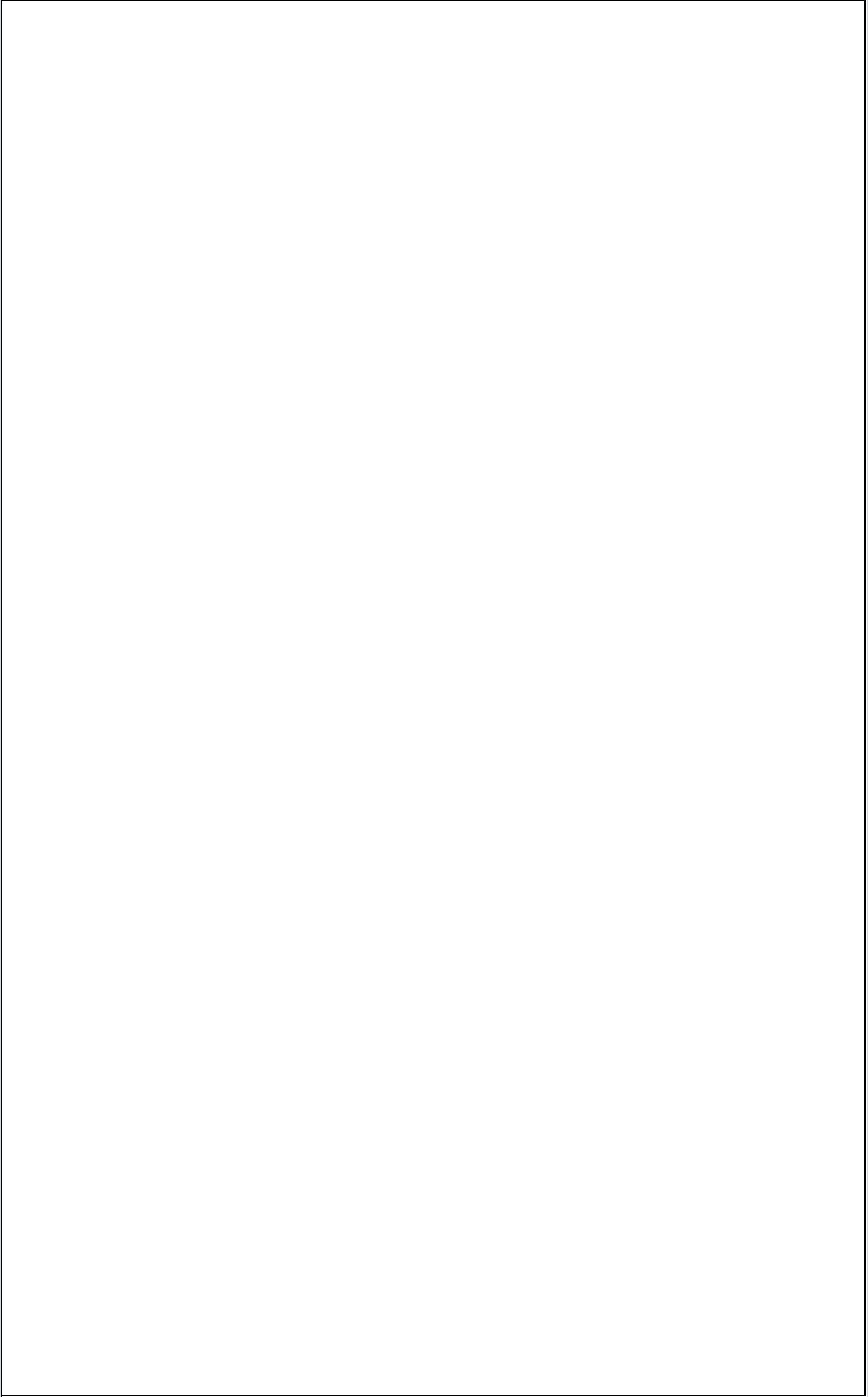
30
31
32
33
34
35
36
37

38

39
40

41
42
43
44

45
46
47



Error

TEST BLOCKED The reactivation flow could not be executed because no sleep/wake control was found on the app.

Observations: - The app loaded its main checker interface and demo dataset (dashboard and data preview visible). - The Column Mapping section shows "Demo columns mapped automatically." and is already present. - No 'Wake' button or a "This app is sleeping" message was found after multiple waits and scrolling.

Cause

The hosting side likely has the app in an always-awake or already-awake state, so the platform never exposes the sleep/reactivation UI. This can happen if the service is configured to stay active, if recent traffic/background probes prevent hibernation, if a custom proxy or domain bypasses the platform's sleep screen, or if the app is deployed on a host where Streamlit's sleep/wake flow is not supported. As a result, the test cannot find a Wake button or sleeping message and is blocked.

Fix

Ensure the Streamlit app is actually deployed in a sleeping/hibernating host state when idle, or disable the inactivity sleep behavior entirely. If using Streamlit Community Cloud or a similar platform, verify the app is not being kept permanently active by background traffic, health checks, or a misconfigured always-on setting. Confirm the correct app/version is deployed and that the host is not bypassing the sleep screen due to a reverse proxy, custom domain, or cached static landing page. If the intended reactivation test depends on a sleep state, adjust hosting so the sleep/wake control is exposed again and the app returns the standard 'This app is sleeping' prompt with a Wake button after inactivity.